This is important: if you're sorting a large set of data, spawning a new thread for each recursion would quickly result in a lot of threads. As you'll see when we look at performance, if you have too many threads, you might actually *slow down* the application. There's also a possibility of running out of threads if the data set is very large. The idea of dividing the overall task in a recursive fashion like this is a good one; you just need to keep a tighter rein on the number of threads. `std::async()` can handle this in simple cases, but it's not the only choice.

One alternative is to use the `std::thread::hardware_concurrency()` function to choose the number of threads, as you did with the parallel version of `accumulate()` from listing 2.8. Then, rather than starting a new thread for the recursive calls, you can just push the chunk to be sorted onto a thread-safe stack such as one of those described in chapters 6 and 7. If a thread has nothing else to do, either because it has finished processing all its chunks or because it's waiting for a chunk to be sorted, it can take a chunk from the stack and sort that.

The following listing shows a sample implementation that uses this technique.

Listing 8.1. Parallel Quicksort using a stack of pending chunks to sort

```
template<typename T>
struct sorter                                    <-- (1)
{
    struct chunk_to_sort
    {
        std::list<T> data;
        std::promise<std::list<T> > promise;
    };

    thread_safe_stack<chunk_to_sort> chunks;     <-- (2)
    std::vector<std::thread> threads;            <-- (3)
    unsigned const max_thread_count;
    std::atomic<bool> end_of_data;

    sorter():
        max_thread_count(std::thread::hardware_concurrency()-1),
        end_of_data(false)
    {}
```